

159.735 Parallel Programming

More notes on GPGPU and CUDA

What are GPUs?

Graphical Processor Units.

- A GPU (on a graphics card) offloads/accelerates graphics rendering from a CPU.
- Modern GPU functions:
 - rendering polygons
 - texture mapping
 - coordinate transformations
 - accelerated video decoding
- Manufacturers
 - NVIDIA
 - ATI

GPGPU

- General Purpose Computing on Graphical Processing Units
 - using a GPU for applications traditionally handled by a CPU
- Stream Processing
 - stream of data
 - a series of operations applied to that stream—the kernel functions
- Recall SPMD
 - single program, multiple data
 - related to, but not the same as SIMD

Programming GPUs

- Approach is to make use of the GPU AND the CPU CUDA
 - Compute Unified Device Architecture
 - Developed and distributed by NVIDIA
- OpenCL
 - tedious and not as good performance as CUDA (according to NVIDIA)

- OpenMP
 - has support for GPU programming
 - a recent development

GPU architecture

Physical layout varies among GPU makes and models, but follows these general ideas:

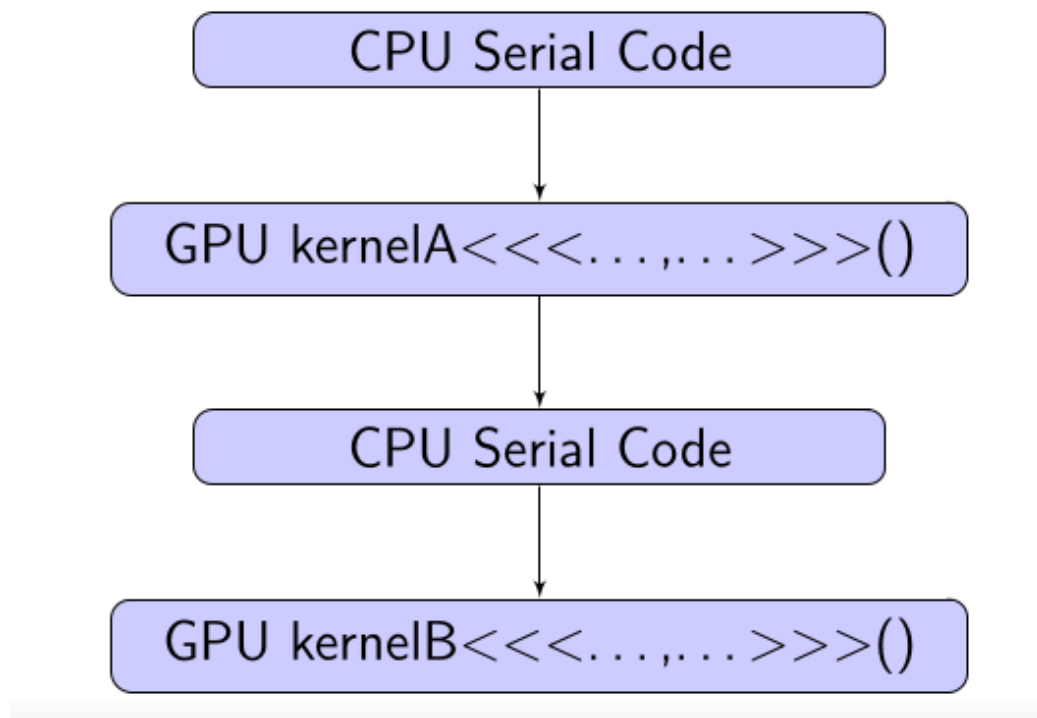
- GPU divided into blocks
- Each block contains one or more streaming multiprocessors
- Each SM has a number of streaming processors
 - all share the same control logic and instruction cache within an SM
- All SPs from all SMs have access to up to several GB GDDR DRAM global memory
 - GDDR: graphics double data rate
 - DRAM: dynamic random access memory

Tesla T4 Card



CUDA processing flow

Need to identify those parts of the program that operate on the host (CPU) and the device (GPU).



See CUDA demos provided on Stream

Important constructs

- Important functions

```
cudaMalloc(device_address, size);  
cudaMemcpy(dest, source, size, cudaMemcpyHostToDevice)  
cudaMemcpy(dest, source, size, cudaMemcpyDeviceToHost)
```

- Function modifier keywords

```
__global__    // called by host, executed on device  
__device__    // called by and executed on device  
__host__      // called by and executed on host
```

- Kernel functions—code to be run on a stream processor

```
mykernel<<<blocks_per_grid, threads_per_block>>>
```

Programming abstractions

- host (CPU) and device (GPU)
- thread
 - concurrent code executed on an SP
 - fine grain unit of parallelism
- warp
 - group of threads executed in parallel (up to a maximum number 32)
- block
 - group of threads executed together and form the unit of resource assignment
- grid
 - group of thread blocks that must all complete before control is returned to the host

Organizing threads

- Threads in a warp can share instruction stream
- Each thread has its own registers and local memory
- Threads in a block can communicate by shared memory
- All threads in a grid can access the same global memory (but with 200–600 cycle penalty)
- Need to decide how many blocks in the grid, and how many threads in each block.
- Can arrange blocks and threads in 1, 2, or 3 dimensions

CUDA Device Memory Types

- GlobalMemory
 - largest memory on GPU and accessible by all threads
 - slowest access time: $\sim 200\text{--}600$ clock cycles
 - lifetime: application
- Registers
 - fastest memory, used to store local variables of a single thread
 - lifetime: thread

- Local memory
 - section of device memory used when variables of a thread do not fit the registers available
 - lifetime: thread
- Shared memory
 - fast on chip memory shared between all threads of a single block
 - lifetime: block
- Texture memory
 - a cached region of global memory
 - each SM has its own texture memory cache on chip
 - lifetime: application
- Constant memory
 - a cached read-only region of device memory on each SM
 - lifetime: application

Performance

- GPU bundles several threads together for execution into “warps”
- Thread index values within a warp are contiguous. For warp size of 32 we have
 - `threadIdx.x` 0 → 31 in warp 0
 - `threadIdx.x` 32 → 63 in warp 1
 - ...
- Bit more complicated for multidimensional thread organization

Branching

- Single Instruction Multiple Thread
 - executes instruction for all threads in the warp, before moving onto next instruction
- Divergence occurs when threads in a warp follow different control flows. Sequential passes are then needed which can affect performance
- Also be careful of conditionals based on thread ID

Memory access

- DRAM memory access patterns
 - Fast: accessing data from multiple and contiguous locations – Slow: truly random access
- Ideal access pattern in GPUs
 - all threads in a warp access consecutive global memory locations
- Coalescing memory access
 - hardware can combine all of these accesses into a single request
- Non-coalescing memory access affects performance